

P3820R0

Split **constexpr** **uncaught_exceptions** into distinct runtime and consteval functions

Published Proposal, 2025-08-31

Author:

[Lénárd Szolnoki](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

1 Background

2 Proposal

3 Motivation

4 Implementation experience

5 Wording

5.1 [exception.syn]

5.2 [uncaught.exceptions]

References

Informative References

§ 1. Background

[P3068] applied the `constexpr` specifier to the functions in the header `<exception>`.

`uncaught_exceptions` is one of these functions, however at runtime its return value depends on mutable global state. It is impossible to implement to return a `constexpr` value that is consistent with the runtime state. When evaluated in a constant expression, the current implementations of [P3068] return the number of uncaught exceptions in the current constant evaluation instead of the current thread.

The inconsistency of the semantics of `uncaught_exceptions` depending on whether it is called from a constant expression or at runtime causes a breaking change when the function is called in trial constant evaluation.

`const` integer variables are usable in constant expressions when they are constant-initialized, so when initializing them the initializer is constant evaluated first, and if that fails then the variable is initialized at runtime. Consider:

```
#include <exception>

int i = 1;

int main () {
    try {
        struct S {
            ~S() {
                // x is initialized to 0 in trial constant evaluation
                const int x = std::uncaught_exceptions();
                i = x;
            }
        };
        S s;
        throw 0;
    } catch (...) {
        return i;
    }
}
```

The program returns 0, which is a breaking change compared to C++23, where it returns 1.

§ 2. Proposal

1. Remove the `constexpr` specifier from `uncaught_exceptions()`.
2. Add a `constexpr int consteval_uncaught_exceptions() noexcept;` function with similar semantics to `uncaught_exceptions()`, but counting exceptions within the current constant evaluation.

§ 3. Motivation

`uncaught_exceptions()` is useful to write scope guards and other patterns to conditionally execute code when normally exiting a scope or due to stack unwinding. Now that we have exception facilities available in constant evaluation, this functionality is similarly useful in that context. Putting the functionality in a separate `constexpr` function makes this functionality available in constant evaluation without a breaking change.

§ 4. Implementation experience

There is previous implementation experience for [P3068] including `constexpr int uncaught_exceptions()`. Splitting the functionality into separate runtime and `constexpr` functions is a trivial change.

§ 5. Wording

Wording is relative to [N5013].

§ 5.1. [exception.syn]

```
constexpr int uncaught_exceptions() noexcept;  
constexpr int consteval_uncaught_exceptions() noexcept;
```

§ 5.2. [uncaught.exceptions]

```
constexpr int uncaught_exceptions() noexcept;
```

Returns: The number of uncaught exceptions ([except.throw]) in the current thread.

Remarks: When `uncaught_exceptions()` > 0 , throwing an exception can result in a call of the function `std::terminate`.

`constexpr int constexpr_uncaught_exceptions() noexcept;`

Returns: The number of uncaught exceptions (`[except.throw]`) in the current constant evaluation.

Remarks: When `constexpr_uncaught_exceptions()` > 0 , throwing an exception can result in a call of the function `std::terminate`.

§ References

§ Informative References

[N5013]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. URL: <https://www.open-std.org/jtc1/sc22/wg21/prot/14882fdis/n5013.pdf>

[P3068]

Hana Dusíková. *Allowing exception throwing in constant-evaluation*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3068r6.html>